

Agentic Layer — Design Doc

Authors	Annie Zhang	Status	PROPOSAL
Last Updated	July 5, 2026	Reviewers	Platform Infrastructure Team, Security Lead

1. Context & Objectives

Abstract

This document proposes adding a conversational and autonomous "Swim Coach" agent layer to the UBC Length Swim application. The agent will orchestrate the application's existing capabilities to turn click-driven actions into a natural-language interface without rewriting the underlying core application services.

Background

Today, the application is a solid full-stack CRUD product consisting of a React SPA over a Spring Boot REST API. It ingests the UBC Aquatic Centre schedule, allows users to log swims, and ranks a daily leaderboard. Currently, all user interactions are strictly click-driven, requiring navigation across multiple separate tabs. The client application is built as a minimal, high-performance SPA using React, TypeScript, and Vite with Hot Module Replacement (HMR) and strict linting constraints.

Goals (In-Scope)

- **Natural Language Orchestration:** Allow users to execute complex, multi-step actions (e.g., finding a specific lane time and generating a matching workout template) via a single conversational prompt.
- **Zero Core Modification:** Adhere to the core design principle: The agent gets no new powers — it orchestrates the tools you already have. Every new capability must be implemented as a thin wrapper over existing domain services (UbcFeedService, SwimRecordRepository, leaderboard logic, user profile).
- **Maintain Build Independence:** Keep the agent entirely isolated inside a new package within the existing Spring Boot backend, gated behind a feature flag (app.agent.enabled=false) so the app can compile, build, and deploy smoothly even if the agentic layer is deactivated.

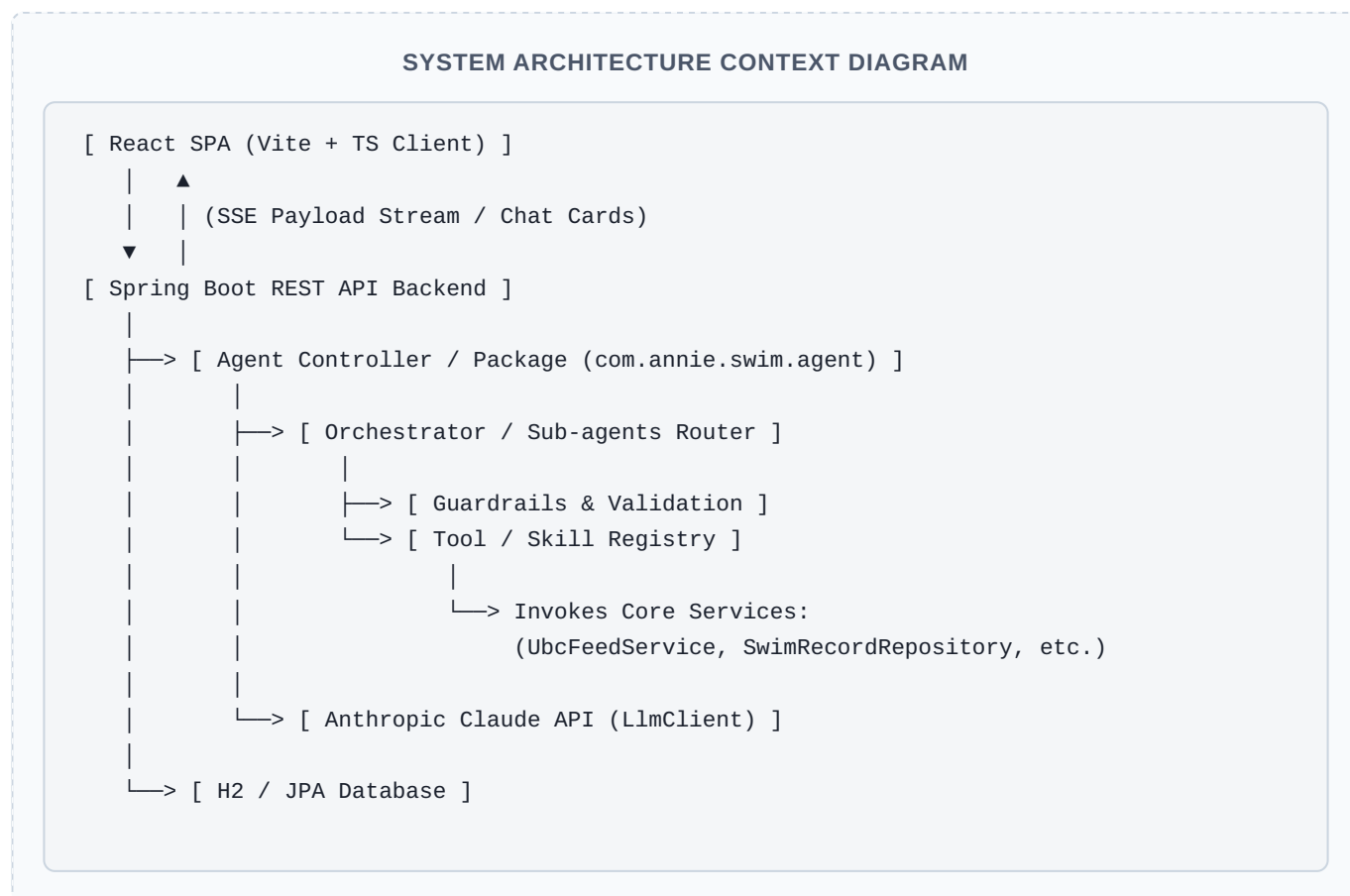
Non-Goals (Out-of-Scope)

- **Enabling the React Compiler:** The React Compiler will remain disabled in the build configuration for this phase due to its current impact on development and build performance.
- Building an internal email or push-notification engine.
- Writing safety-critical, medical, or diagnostic coaching algorithms (advice will remain strictly restricted to general fitness).
- Automating physical lane bookings without explicit user confirmation screens in the UI (skills will remain read-mostly in early phases).

2. Proposed Architecture

High-Level System Diagram

The agent lives inside the existing Spring Boot backend as one new package (`com.annie.swim.agent`). It communicates with the React + TypeScript front-end client over Server-Sent Events (SSE).



Detailed Component Breakdown

- **AgentController (POST /api/agent/chat):** Exposes the execution endpoint to the UI and establishes an SSE (Server-Sent Events) pipeline to stream partial response tokens and interactive structured cards.
- **AgentOrchestrator:** Manages the lifecycle loop of execution (Perceive → Reason → Act → Observe).
- **Tool/Skill Registry:** Dynamically auto-discovers Spring bean capabilities implementing the uniform Skill interface, compiling metadata (JSON schemas) to pass to the LLM.
- **Router & Sub-agents:** Classifies incoming user intent and delegates execution to specialized sub-agent layers (e.g., Coach, Analyst, Scheduler) to minimize system prompt inflation and increase deterministic reliability.
- **Frontend Client Integration:** Extends the existing React codebase. The client-side streaming UI uses `@vitejs/plugin-react` (via `Oxc`) or `@vitejs/plugin-react-swc` for lean local rendering and compilation.

Data Model & Schema Changes

Four new tables will be added to the database to support conversational history, autonomous background state tracking, and deterministic debugging:

Table Name	Purpose / Description
conversation	Tracks single chat threads mapped to specific userId references.
conversation_message	Persists individual chat history turns (user messages, assistant text).
agent_step	Detailed audit log tracking intermediate reasoning plans, tool execution parameters, observations, token usage, and latency metrics.
scheduled_agent	Manages background cron execution configurations for user subscriptions (e.g., automated weekly digests).

API / Interface Definitions

Every skill capability must implement the following base Java interface configuration to be automatically gathered by the registry:

```
public interface Skill {
    String name(); // e.g., "schedule.find_sessions"
    String description(); // Fed directly to the LLM for tool selection context
    JsonSchema parameterSchema(); // Validated before execute() runs
    SkillResult execute(SkillContext ctx, JsonNode args);
}
```

3. Alternatives Considered

Alternative A: Move Agent Logic Entirely to Frontend Client (NextJS Edge/React Components)

- **Pros:** Lower backend state utilization; direct streaming integration with client browser layout contexts.
- **Cons:** Exposes database schema details, sensitive internal system prompts, and direct LLM API access credentials to the public client environment. It also prevents running autonomous background processing cron jobs (like the Briefing sub-agent) when the user closes their browser window.
- **Decision:** Rejected. Keeping the agent server-side inside Spring Boot preserves standard backend isolation, enables securely guarded access to DB models, and supports scheduled background worker threads.

Alternative B: Single Master Agent System Prompt (No Sub-Agents)

- **Pros:** Straightforward implementation path; zero context translation layer overhead between components.

- **Cons:** As more tools are registered, token consumption increases exponentially. Massive prompt contexts degrade reasoning capability, resulting in frequent tool hallucinations or broken argument blocks.
- **Decision:** Rejected in favor of an Orchestrator/Router pattern that safely partitions prompts and toolsets into isolated sub-agent scopes.

4. Cross-Cutting Concerns

Scalability & Operational Budgets

LLM inferences are non-deterministic, high-latency, and carry financial token costs. The system will mitigate this through:

- **Tool Execution Caching:** Short-term lookups (such as fixed calendar schedules) will be cached within the scope of a single conversational turn.
- **Step Budgets:** A hard-capped ceiling will govern the absolute number of allowed tool execution loops per prompt turn to avoid runaway loops.

Reliability & Guardrails

A dedicated validation filter wraps every processing cycle execution step:

- **Argument Validation:** Input strings from the LLM are evaluated directly against the skill's registered JsonSchema parameters before executing code.
- **Safety Boundaries:** Active runtime filtering handles content safety bounds, domain verification checks, and age-aware restrictions.

Observability & Code Quality

- **Backend Transparency:** The agent_step table acts as a built-in observability repository to capture and replay broken execution flows deterministically.
- **Frontend Type Safety & Linting:** To prevent interface desynchronization with the streaming SSE payload, the React client's ESLint configuration is expanded to force strict, type-aware lint rules (tseslint.configs.strictTypeChecked and tseslint.configs stylisticTypeChecked) targeting all `**/*.{ts,tsx}` files. React-specific runtime safety is reinforced using eslint-plugin-react-x and eslint-plugin-react-dom.

5. Implementation & Rollout Plan

Milestones

The engineering rollout will follow a phased delivery lifecycle:

1. **Phase 1 (MVP):** Deploy single-agent execution system backend alongside ScheduleSkill and ProgressSkill capabilities. Upgrade the React + TypeScript frontend client structure with the strict ESLint configurations (tsconfig.app.json / tsconfig.node.json parsing rules) and introduce the new Coach interface chat tab supporting SSE token streaming.

2. **Phase 2 (v2):** Integrate the Orchestrator/Router system with sub-agent boundaries. Introduce WorkoutSkill, activate robust input validation schemas, and build the automated @Scheduled background briefing framework.
3. **Phase 3 (v3):** Expose the inner tool registry via a formal Model Context Protocol (MCP) server adapter configuration, converting internal app functionality into a platform consumable by external development clients (e.g., Claude Desktop, Cursor).

Rollback Strategy

If runtime latency spikes or message queue drops occur, the entire layer can be deactivated globally by toggling the server configuration file flag `app.agent.enabled=false`. This safely unloads the agent package, while the core React tabs continue running over Vite with HMR without relying on any LLM endpoints.